

Lösung Praxisaufgabe 4

3 Rechen-Server

3.1 Verwendete Python-Programme und Ausführung

Für die Versuche wurden mehrere Python-Programme verwendet. Ich habe die Programme jeweils so ausgeführt, dass die Konsolenausgabe mit `tee` in eine Logdatei geschrieben wurde. Dadurch sind die Versuche wiederholbar und die Ausgaben können später direkt in der Auswertung verwendet werden. Die Option `-u` wurde bei Serverprogrammen verwendet, damit Python die Ausgabe nicht puffert. Bei allen Befehlen leitet `2>&1` auch Fehlermeldungen in die Logdatei um.

calc_server_tcp_threaded.py

Dieses Programm ist der TCP-Rechenserver. Es bindet einen Stream-Socket an eine IP-Adresse und einen Port, wartet mit `accept()` auf eingehende Verbindungen und startet für jeden Client einen eigenen Thread. In diesem Thread wird die Anfrage empfangen, geparkt, berechnet und beantwortet. Lokal wurde der Server mit folgendem Befehl gestartet:

```
python3 -u calc_server_tcp_threaded.py --ip 127.0.0.1 --port 50001 \
  2>&1 | tee log_calc_server_tcp_local_clean.txt
```

Für den Hotspot-Versuch wurde derselbe Server auf die Hotspot-IP des Fedora-Laptops gebunden:

```
python3 -u calc_server_tcp_threaded.py --ip 172.20.10.2 --port 50001 \
  2>&1 | tee log_calc_server_tcp_hotspot_clean.txt
```

calc_server_udp.py

Dieses Programm ist der UDP-Rechenserver. Es bindet einen Datagram-Socket, empfängt Anfragen mit `recvfrom()`, berechnet das Ergebnis und sendet die Antwort mit `sendto()` an den Absender zurück. Ausgeführt wurde das Programm mit:

```
python3 -u calc_server_udp.py --ip 127.0.0.1 --port 50002 --seconds 60
  2>&1 | tee log_calc_server_udp_local_clean.txt
```

calc_client_probe.py

Dieses Programm wurde als Client für TCP und UDP verwendet. Es erzeugt die binäre Rechenanfrage mit `struct.pack()`, sendet sie an den Server, gibt die lokale Socket-Adresse und die Gegenstelle aus und parst danach die Antwort. Mit demselben Programm wurde auch untersucht, ob der lokale Client-Port automatisch vergeben wird oder mit `bind()` gesetzt werden kann.

Ein lokaler TCP-Test wurde zum Beispiel so gestartet:

```
python3 calc_client_probe.py --proto tcp --server-ip 127.0.0.1 \
  --server-port 50001 --op SUM --numbers 1 2 3 4 5 \
  2>&1 | tee log_calc_client_tcp_sum_clean.txt
```

Der UDP-Test wurde so gestartet:

```
python3 calc_client_probe.py --proto udp --server-ip 127.0.0.1 \
  --server-port 50002 --op SUM --numbers 1 2 3 4 5 \
  2>&1 | tee log_calc_client_udp_sum_clean.txt
```

Im Hotspot-Versuch ohne explizites lokales Binding wurde folgender Befehl verwendet:

```
python3 calc_client_probe.py --proto tcp --server-ip 172.20.10.2 \
  --server-port 50001 --op SUM --numbers 1 2 3 4 5 \
  2>&1 | tee log_calc_client_tcp_hotspot_auto_port.txt
```

Im Hotspot-Versuch mit explizitem lokalen Port wurde folgender Befehl verwendet:

```
python3 calc_client_probe.py --proto tcp --server-ip 172.20.10.2 \
  --server-port 50001 --bind-ip 172.20.10.4 --bind-port 50010 \
  --op SUM --numbers 1 2 3 4 5 \
  2>&1 | tee log_calc_client_tcp_hotspot_bound_port.txt
```

echo_dual_tcp_udp_server.py und echo_client.py

Mit `echo_dual_tcp_udp_server.py` wurde getestet, ob TCP und UDP dieselbe numerische Portnummer gleichzeitig nutzen können. Das Programm bindet dazu einen TCP-Socket und einen UDP-Socket auf `127.0.0.1:50000`. Beide Dienste senden empfangene Daten wieder zurück.

```
python3 -u echo_dual_tcp_udp_server.py --ip 127.0.0.1 --port 50000 \
  --seconds 60 2>&1 | tee log_echo_dual_server_clean.txt
```

Die Tests wurden mit `echo_client.py` durchgeführt. Der TCP-Test war:

```
python3 echo_client.py --proto tcp --server-ip 127.0.0.1 --server-port  
--message "Hello TCP" 2>&1 | tee log_echo_tcp_same_port_clean.txt
```

Der UDP-Test war:

```
python3 echo_client.py --proto udp --server-ip 127.0.0.1 --server-port  
--message "Hello UDP" 2>&1 | tee log_echo_udp_same_port_clean.txt
```

port_scanner_tcp_udp.py

Dieses Programm ist der Portscanner. Für TCP wird mit `connect_ex()` geprüft, ob ein Verbindungsaufbau möglich ist. Für UDP sendet das Programm ein Datagramm und wartet dann auf eine Antwort, einen Fehler oder einen Timeout. Die einzelnen Portanfragen laufen parallel.

Der TCP-Scan wurde mit folgendem Befehl ausgeführt:

```
python3 port_scanner_tcp_udp.py --proto tcp --host 141.37.122.107 \  
--start 1 --end 50 --timeout 1 --workers 50 \  
2>&1 | tee log_scan_tcp_141_37_122_107.txt
```

Der UDP-Scan wurde mit folgendem Befehl ausgeführt:

```
python3 port_scanner_tcp_udp.py --proto udp --host 141.37.168.26 \  
--start 1 --end 50 --timeout 1 --workers 50 \  
2>&1 | tee log_scan_udp_141_37_168_26.txt
```

SMTP-Programme

Für den OpenSSL-Versuch wurde kein eigenes Python-Programm benutzt. Der Socket wurde direkt mit `openssl s_client` geöffnet:

```
openssl s_client -starttls smtp -crlf -connect asmtplib.htwg-konstanz.de:587
```

Für den Python-Versuch wurde `smtp_socket_client.py` verwendet. Das Programm öffnet zuerst einen TCP-Socket zum SMTP-Server auf Port 587, sendet EHLO und STARTTLS, wandelt den Socket mit `ssl.create_default_context()` in einen TLS-Socket um und führt danach den SMTP-Dialog mit AUTH LOGIN, mail from, rcpt to und data aus.

```
python3 smtp_socket_client.py --username pa871kru \  
--from-envelope pa871kru@htwg-konstanz.de \  
--from-header "Paul Kruessel <pa871kru@htwg-konstanz.de>" \  
--to kruesselpaul@gmail.com \  
--subject "RN Labor Python SMTP Test" \  
--body "Testmail per Python-Socket" \  
2>&1 | tee log_smtp_python.txt
```

3.2 Lokale Kommunikation

Für den lokalen Test des Rechenservers wurden Client und Server auf demselben Rechner ausgeführt. Als Serveradresse wurde die Loopback-Adresse 127.0.0.1 verwendet. Der Server lauschte auf Port 50001. Dadurch findet die Kommunikation vollständig lokal über die Loopback-Schnittstelle statt.

3.2.1 Versuchsaufbau

Zunächst wurde das Server-Skript gestartet. Der Server erzeugt einen TCP-Socket, bindet diesen an die Adresse 127.0.0.1:50001 und wartet anschließend auf eingehende Verbindungen:

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(("127.0.0.1", 50001))
sock.listen(5)
conn, addr = sock.accept()
```

Danach wurde das Client-Skript gestartet. Der Client erzeugt ebenfalls einen TCP-Socket und verbindet sich mit dem Server:

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("127.0.0.1", 50001))
```

Als Testanfrage wurde die Operation SUM mit den Zahlen 1, 2, 3, 4, 5 verwendet. Die Task-ID war 5000. Das erwartete Ergebnis ist somit:

$$1 + 2 + 3 + 4 + 5 = 15$$

3.2.2 Nachrichtenformat

Die Anfrage des Clients hat nach Aufgabenstellung folgendes Format:

$$< ID > < Operation > < N > < z_1 > < z_2 > \dots < z_N >$$

Dabei wird die ID als unsigned Integer mit 4 Byte übertragen. Die Rechenoperation wird als UTF-8-Zeichenkette übertragen. Die Anzahl der Zahlen wird als unsigned char mit 1 Byte übertragen. Die Zahlen selbst werden jeweils als signed Integer mit 4 Byte übertragen.

Für den Testfall ergibt sich folgende Nachricht:

Feld	Wert	Kodierung	Länge
ID	5000	unsigned int	4 Byte
Operation	SUM	UTF-8	3 Byte
N	5	unsigned char	1 Byte
z_1	1	signed int	4 Byte
z_2	2	signed int	4 Byte
z_3	3	signed int	4 Byte
z_4	4	signed int	4 Byte
z_5	5	signed int	4 Byte

Die vom Client erzeugte Bytefolge war:

```
b'\x00\x00\x13\x88SUM\x05
\x00\x00\x00\x01
\x00\x00\x00\x02
\x00\x00\x00\x03
\x00\x00\x00\x04
\x00\x00\x00\x05'
```

In hexadezimaler Schreibweise entspricht dies:

```
0000138853554d050000000100000002000000030000000400000005
```

Die Antwort des Servers hat das Format:

< ID > < Ergebnis >

Für die ID 5000 und das Ergebnis 15 wurde folgende Antwort empfangen:

```
000013880000000f
```

Dabei entspricht 00001388 der Task-ID 5000 und 0000000f dem dezimalen Ergebnis 15.

3.2.3 Programmausgabe

Beim Ausführen des Clients wurde folgende Ausgabe erzeugt:

```
Parsed Message: b'\x00\x00\x13\x88SUM\x05
\x00\x00\x00\x01\x00\x00\x00\x02\x00\x00\x00\x03
\x00\x00\x00\x04\x00\x00\x00\x05'
sent: 000013880000000f
received: 000013880000000f
(5000, 15)
```

Die Ausgabe zeigt, dass der Client die Antwort des Servers korrekt empfangen und geparkt hat. Die Antwort enthält die ursprüngliche Task-ID 5000 und das Ergebnis 15.

3.2.4 Beobachtung in Wireshark

Die Kommunikation wurde mit Wireshark auf der Loopback-Schnittstelle aufgezeichnet. Bei der TCP-Kommunikation waren zunächst die Pakete des TCP-Verbindungsaufbaus zu sehen. Danach wurde die eigentliche Nutzlast übertragen. Anschließend sendete der Server die Antwort an den Client zurück. Die beobachtete Paketfolge lässt sich wie folgt erklären:

Paket	Richtung	Bedeutung	Verantwortlicher Python-Befehl
SYN	Client → Server	Beginn des TCP-Verbindungsaufbaus	<code>sock.connect(...)</code> im Client
SYN, ACK	Server → Client	Bestätigung der Verbindungsanfrage	Betriebssystem des Servers nach <code>listen(...)</code>
ACK	Client → Server	Abschluss des Three-Way-Handshakes	Betriebssystem des Clients
PSH, ACK	Client → Server	Übertragung der Rechenanfrage	<code>sock.send(...)</code> im Client
PSH, ACK	Server → Client	Übertragung des Ergebnisses	<code>conn.sendall(...)</code> im Server
FIN, ACK	Client → Server	Verbindungsabbau	<code>sock.close()</code> im Client
FIN, ACK	Server → Client	Bestätigung des Verbindungsabbaus	<code>conn.close()</code> im Server

3.2.5 Zuordnung der blockierenden Befehle

Mehrere Socket-Befehle blockieren, bis bestimmte Netzwerkereignisse eintreten. Im Versuch waren insbesondere `connect`, `accept` und `recv` relevant.

Befehl	Skript	Wird fortgesetzt durch
<code>sock.connect(...)</code>	Client	Abschluss des TCP-Verbindungsaufbaus durch SYN, SYN-ACK und ACK
<code>sock.accept()</code>	Server	Eintreffen einer neuen TCP-Verbindungsanfrage vom Client
<code>conn.recv(...)</code>	Server	Eintreffen der vom Client gesendeten Rechenanfrage
<code>sock.recv(...)</code>	Client	Eintreffen der Serverantwort mit ID und Ergebnis

Der Befehl `accept()` im Server wartet so lange, bis ein Client eine Verbindung aufbaut. Der Befehl `recv()` im Server wartet anschließend auf die Nutzdaten des Clients. Im Client blockiert `recv()`, bis der Server das Ergebnis der Berechnung zurückgeschickt hat.

3.2.6 Test mit Telnet

Zusätzlich wurde der Server mit Telnet getestet. Der Aufruf lautet:

```
telnet 127.0.0.1 50001
```

Damit kann eine TCP-Verbindung zum Server aufgebaut werden. Telnet eignet sich, um grundsätzlich zu prüfen, ob der Server auf dem angegebenen Port erreichbar ist. Für die eigentliche Rechenanfrage ist Telnet jedoch nur eingeschränkt geeignet, da das Protokoll binäre Daten erwartet. Die Anfrage enthält beispielsweise Integer-Werte im Binärformat, die sich über die Tastatur nicht sinnvoll direkt eingeben lassen.

Der Telnet-Test zeigt daher vor allem, dass der TCP-Port geöffnet ist und der Server Verbindungen annimmt. Die korrekte Berechnung wurde mit dem Python-Client geprüft, da dieser das geforderte Binärformat mit `struct.pack()` erzeugt.

3.2.7 Ergebnis

Der lokale Test war erfolgreich. Der Client konnte über TCP eine Verbindung zum Server unter `127.0.0.1:50001` aufbauen, eine korrekt kodierte Rechenanfrage senden und die Antwort empfangen. Für die Anfrage

$$SUM(1, 2, 3, 4, 5)$$

lieferte der Server korrekt das Ergebnis

15

zurück. Die empfangene Antwort

```
000013880000000f
```

enthält die ursprüngliche Task-ID 5000 sowie das Ergebnis 15. Damit entspricht die Kommunikation dem vorgegebenen Nachrichtenformat.

3.3 Netzwerk-Kommunikation

Für den Versuch über ein Netzwerk wurde nicht NordVPN MeshNet verwendet. Stattdessen wurde ein Handy-Hotspot genutzt, sodass beide Geräte im gleichen lokalen Netz waren. Der Server lief auf dem Fedora-Laptop mit der IP-Adresse `172.20.10.2`. Der Client lief auf einem Windows-Gerät mit der IP-Adresse `172.20.10.4`. Vor dem eigentlichen Test wurde die Verbindung mit `ping` geprüft. Die dazu verwendeten Programme und Befehle sind im Abschnitt zu den verwendeten Python-Programmen aufgeführt.

Ping wird ausgeführt für 172.20.10.2 mit 32 Bytes Daten:

Antwort von 172.20.10.2: Bytes=32 Zeit=4ms TTL=64

Antwort von 172.20.10.2: Bytes=32 Zeit=4ms TTL=64

Antwort von 172.20.10.2: Bytes=32 Zeit=3ms TTL=64

Antwort von 172.20.10.2: Bytes=32 Zeit=83ms TTL=64

Ping-Statistik für 172.20.10.2:

Pakete: Gesendet = 4, Empfangen = 4, Verloren = 0
(0% Verlust),

Ca. Zeitangaben in Millisek.:

Minimum = 3ms, Maximum = 83ms, Mittelwert = 23ms

Der Ping-Test zeigt, dass der Server vom Client aus erreichbar war. Es wurden alle vier Pakete beantwortet und es gab keinen Paketverlust.

3.3.1 Lokale Adresse und lokaler Port des Clients

Im ersten Versuch wurde der Client ohne explizites `bind()` gestartet. Der Client verbindet sich also nur mit dem Server. Die lokale Adresse wird dann vom Betriebssystem gewählt.

```
[CLIENT TCP] before bind/connect getsockname=not available yet
[CLIENT TCP] connecting to ('172.20.10.2', 50001)
[CLIENT TCP] after connect local getsockname=('172.20.10.4', 62358)
[CLIENT TCP] after connect peer=('172.20.10.2', 50001)
[CLIENT TCP] TX 28 bytes: 0000138853554d05000000010000000200000000300000
[CLIENT TCP] RX 8 bytes: 0000138800000000f
[CLIENT TCP] parsed response=(5000, 15)
```

Mit `getsockname()` kann man im Client die lokale IP-Adresse und den lokalen Port des Sockets herausfinden. Auf Windows lieferte `getsockname()` vor `connect()` noch einen Fehler, weil der Socket noch nicht gebunden bzw. verbunden war. Nach `connect()` war die lokale Adresse aber bestimmt. In diesem Fall war dies `172.20.10.4:62358`. Der Port 62358 wurde also automatisch vom Betriebssystem vergeben.

Im zweiten Versuch wurde der lokale Socket vor dem `connect()` explizit gebunden:

```
[CLIENT TCP] before bind/connect getsockname=not available yet
[CLIENT TCP] after explicit bind getsockname=('172.20.10.4', 50010)
[CLIENT TCP] connecting to ('172.20.10.2', 50001)
[CLIENT TCP] after connect local getsockname=('172.20.10.4', 50010)
[CLIENT TCP] after connect peer=('172.20.10.2', 50001)
[CLIENT TCP] TX 28 bytes: 0000138853554d05000000010000000200000000300000
```



```
[CLIENT TCP] RX 8 bytes: 000013880000000f
[CLIENT TCP] parsed response=(5000, 15)
```

Hier sieht man, dass der Client lokal auf 172.20.10.4:50010 gebunden wurde. Nach dem Verbindungsaufbau blieb diese lokale Adresse erhalten. Der Client kann seine lokale IP-Adresse und Portnummer also mit folgendem Befehl setzen:

```
sock.bind(("172.20.10.4", 50010))
```

Ohne `bind()` erhält der Client seine lokale IP-Adresse und Portnummer beim Verbindungsaufbau durch `connect()`. Die IP-Adresse wird passend zur Route zum Server gewählt. Der lokale Port wird als freier kurzlebiger Port vom Betriebssystem vergeben. Bei UDP passiert dies spätestens beim ersten `sendto()`.

3.3.2 Timeouts und try/except

Timeouts sind wichtig, da Socket-Befehle wie `accept()`, `connect()`, `recv()` oder `recvfrom()` blockieren können. Ohne Timeout kann ein Programm hängen bleiben, wenn ein Paket verloren geht oder wenn die Gegenstelle nicht antwortet. Im Versuch wurde zum Beispiel ein Timeout von 5 Sekunden gesetzt:

```
socket.setdefaulttimeout(5.0)
```

Mit `socket.setdefaulttimeout()` wird ein gemeinsamer Standard-Timeout für neu erzeugte Sockets gesetzt. Einzelne Sockets können auch mit `sock.settimeout(...)` konfiguriert werden.

Mit `try ... except` kann man solche Fehler abfangen. Der Socket-Befehl steht im `try`-Block. Tritt z.B. ein Timeout auf, wird der Programmfluss im passenden `except`-Block fortgesetzt. Dadurch kann man eine Fehlermeldung ausgeben oder sauber abbrechen, anstatt dass das Programm unkontrolliert endet.

3.3.3 TCP und UDP auf gleicher Portnummer

Für die Frage, ob TCP und UDP auf derselben Portnummer gleichzeitig verwendet werden können, wurde das Programm `echo_dual_tcp_udp_server.py` gestartet. Dieses Programm erzeugt einen TCP-Socket und einen UDP-Socket und bindet beide auf die Adresse 127.0.0.1:50000. Die beiden Tests wurden danach mit `echo_client.py` durchgeführt. Die genauen Befehle sind im Abschnitt zu den verwendeten Python-Programmen angegeben.

Serverausgabe:

```
[TCP ECHO] Listening on ('127.0.0.1', 50000)
[UDP ECHO] Listening on ('127.0.0.1', 50000)
[DUAL ECHO] TCP and UDP are bound to the same numeric port 50000
```

```
[TCP ECHO] accepted ('127.0.0.1', 34746)
[TCP ECHO] RX from ('127.0.0.1', 34746): b'Hello TCP'
[TCP ECHO] closed ('127.0.0.1', 34746)
[UDP ECHO] RX from ('127.0.0.1', 57220): b'Hello UDP'
```

TCP-Client:

```
[TCP CLIENT] before connect local=('0.0.0.0', 0)
[TCP CLIENT] after connect local=('127.0.0.1', 34746) peer=('127.0.0.1', 34746)
[TCP CLIENT] TX: b'Hello TCP'
[TCP CLIENT] RX: b'Hello TCP'
```

UDP-Client:

```
[UDP CLIENT] before send local=('0.0.0.0', 0)
[UDP CLIENT] TX: b'Hello UDP'
[UDP CLIENT] after send local=('0.0.0.0', 57220)
[UDP CLIENT] RX from ('127.0.0.1', 50000): b'Hello UDP'
```

Der Versuch zeigt, dass TCP und UDP auf derselben numerischen Portnummer gleichzeitig betrieben werden können. Das funktioniert, weil TCP-Ports und UDP-Ports getrennte Namensräume sind. `127.0.0.1:50000/TCP` und `127.0.0.1:50000/UDP` sind für das Betriebssystem also verschiedene Endpunkte.

3.4 Unterstützung für mehrere Clients

Der TCP-Server wurde so erweitert, dass er für jede neue Verbindung einen eigenen Thread startet. Die Funktion `listen(sock)` wartet auf neue Verbindungen. Sobald `accept()` einen verbundenen Socket liefert, wird ein neuer Thread mit der Funktion `receive(conn, addr)` gestartet.

Der relevante Ablauf ist:

```
conn, addr = sock.accept()
thread = Thread(target=receive, args=(conn, addr))
thread.start()
```

Die Funktion `receive()` bearbeitet dann die Nachrichten genau dieses Clients. Dadurch kann der Server mehrere Clients gleichzeitig bedienen. Getestet wurde dies mit fünf parallel gestarteten Clients.

```
[CLIENT] test=SUM([1, 2, 3]) task_id=1
[CLIENT TCP] parsed response=(1, 6)
```

```
[CLIENT] test=SUM([2, 2, 3]) task_id=2  
[CLIENT TCP] parsed response=(2, 7)
```

```
[CLIENT] test=SUM([3, 2, 3]) task_id=3  
[CLIENT TCP] parsed response=(3, 8)
```

```
[CLIENT] test=SUM([4, 2, 3]) task_id=4  
[CLIENT TCP] parsed response=(4, 9)
```

```
[CLIENT] test=SUM([5, 2, 3]) task_id=5  
[CLIENT TCP] parsed response=(5, 10)
```

Der Server hat die Verbindungen auf unterschiedlichen lokalen Ports angenommen und jeweils die richtige Antwort zurückgeschickt. Damit ist gezeigt, dass mehrere Clients gleichzeitig verarbeitet werden können.

4 Port Scan

4.1 TCP Port Scanner

Für den TCP-Portscan wurde das Programm `port_scanner_tcp_udp.py` verwendet. Dieses Programm startet für die einzelnen Ports parallele Anfragen. Bei TCP wird mit `connect_ex()` geprüft, ob der Verbindungsaufbau erfolgreich ist. Gescannt wurde der Server `141.37.122.107` auf den Ports 1 bis 50. Die Verbindung zur Hochschule wurde vorher über OpenVPN aufgebaut. Im VPN-Log war zu sehen, dass die Verbindung erfolgreich aufgebaut wurde:

```
Initialization Sequence Completed
```

Ein Ping auf den Server lieferte keine Antwort:

```
4 packets transmitted, 0 received, 100% packet loss
```

Das bedeutet aber nicht automatisch, dass der Server nicht erreichbar ist. ICMP kann blockiert sein. Deshalb wurde die Erreichbarkeit der Dienste mit TCP-Sockets getestet. Der genaue Ausführungsbefehl für den Scanner ist im Abschnitt zu den verwendeten Python-Programmen angegeben. Parallel zum Scan wurde in Wireshark auf dem VPN-Interface `tun0` mitgeschnitten. Als Display-Filter wurde verwendet:

```
ip.addr == 141.37.122.107 && tcp
```

Der TCP-Scan lieferte folgende offenen Ports:

Port	Status	Beobachtung
7	offen	Echo-Dienst, Nutzdaten wurden zurückgesendet
9	offen	Verbindung wurde akzeptiert, keine Nutzdaten empfangen
13	offen	Daytime-Dienst, Datum/Uhrzeit wurde gesendet
19	offen	Chargen-Dienst, viele Testzeichen wurden gesendet
22	offen	SSH-Banner wurde gesendet

Auszug aus dem Log:

```
TCP port 7: open connect_ex=0 payload=b'RN-LAB-ECHO-TEST\r\n'
TCP port 9: open connect_ex=0 payload=b''
TCP port 13: open connect_ex=0 payload=b'Tue May 12 09:15:31 2026\r\n'
TCP port 19: open connect_ex=0 payload=b' ... '
TCP port 22: open connect_ex=0 payload=b'SSH-2.0-OpenSSH_9.6p1 Ubuntu-3
```

4.1.1 Wireshark-Auswertung TCP-Scan

Im Wireshark-Mitschnitt sieht man, dass der Scanner für jeden getesteten TCP-Port versucht, eine Verbindung aufzubauen. Jeder Versuch beginnt mit einem TCP-Paket mit gesetztem SYN-Flag vom Client zum Server. Der Quellport ist dabei ein kurzlebiger lokaler Port des Clients. Der Zielport ist der gerade getestete Port zwischen 1 und 50.

Bei einem offenen Port antwortet der Server mit SYN, ACK. Dadurch ist erkennbar, dass auf diesem Port ein Dienst Verbindungen annimmt. Der Client bestätigt anschließend mit ACK. Danach ist die TCP-Verbindung aufgebaut und das Python-Programm kann Testdaten senden oder Daten vom Server empfangen.

Für Port 7 war im Mitschnitt die typische Folge für einen offenen Echo-Dienst zu sehen:

Paket	Richtung	Bedeutung
SYN	Client → Server:7	Start des TCP-Verbindungsaufbaus durch <code>connect_ex()</code>
SYN, ACK	Server:7 → Client	Port ist offen, der Server akzeptiert die Verbindung
ACK	Client → Server:7	Abschluss des Three-Way-Handshakes
PSH, ACK	Client → Server:7	Der Scanner sendet die Testnachricht RN-LAB-ECHO-TEST
PSH, ACK	Server:7 → Client	Der Echo-Dienst sendet dieselben Nutzdaten zurück
FIN/ACK bzw. RST	Client/Server	Die Verbindung wird wieder beendet

Diese Paketfolge passt zur Programmausgabe, da der Scanner für Port 7 eine erfolgreiche Verbindung und die zurückgesendete Nutzlast protokolliert hat. Bei

Port 13 und Port 22 war die Verbindung ebenfalls offen, aber das Verhalten des Dienstes war anders. Port 13 sendete von sich aus eine Datum-/Uhrzeit-Zeile. Port 22 sendete ein SSH-Banner. Daran sieht man, dass ein offener TCP-Port nicht immer auf die vom Client gesendeten Testdaten antworten muss. Manche Dienste senden direkt ein Banner, andere warten auf ein Protokollkommando. Bei geschlossenen Ports war im Mitschnitt keine vollständige TCP-Verbindung zu sehen. Statt eines SYN, ACK kam entweder ein Paket mit RST, ACK zurück oder es kam bis zum Timeout keine Antwort. Ein RST, ACK bedeutet, dass der Zielhost erreichbar ist, auf diesem Port aber kein Dienst lauscht. Ein Timeout bedeutet dagegen, dass keine verwertbare Antwort kam. Das kann an Filterregeln, Firewalls oder Paketverlust liegen. Deshalb wurden solche Ports im Programm als `closed_or_filtered` ausgegeben und nicht weiter unterschieden.

4.2 UDP Port Scanner

Für UDP wurde ebenfalls `port_scanner_tcp_udp.py` verwendet. In diesem Modus sendet das Programm ein UDP-Datagramm an jeden Port und wartet danach auf eine Antwort oder einen Fehler. Gescannt wurde der Server 141.37.168.26 auf den Ports 1 bis 50. Parallel wurde in Wireshark auf dem VPN-Interface `tun0` mitgeschnitten. Der verwendete Display-Filter war:

```
ip.addr == 141.37.168.26 && (udp || icmp)
```

In den Logs wurde für keinen der gescannten UDP-Ports eine Antwort empfangen. Alle Ports liefen in einen Timeout.

```
[SCAN] proto=udp host=141.37.168.26 ports=1-50 timeout=1.0s workers=50
1;os_error_probably_closed;TimeoutError: timed out;;
2;os_error_probably_closed;TimeoutError: timed out;;
...
50;os_error_probably_closed;TimeoutError: timed out;;
[SUMMARY]
```

4.2.1 Wireshark-Auswertung UDP-Scan

UDP unterscheidet sich deutlich von TCP. Bei UDP gibt es keinen Verbindungsaufbau und damit auch keinen Three-Way-Handshake. Im Mitschnitt sieht man daher pro getesteten Port zunächst nur ein einzelnes UDP-Datagramm vom Client zum Server. Das Datagramm enthält die Testnachricht des Scanners. Danach wartet das Python-Programm mit `recvfrom()` auf eine Antwort. Für einen UDP-Port gibt es drei typische Möglichkeiten:

Beobachtung in Wireshark	Bedeutung	Auswirkung im Programm
UDP-Antwort vom Server	Ein Dienst hat auf die Anfrage geantwortet. Der Port ist dadurch experimentell als offen erkennbar.	<code>recvfrom()</code> liefert Daten zurück.
ICMP Destination Unreachable / Port Unreachable	Der Zielhost meldet, dass auf diesem UDP-Port kein Dienst erreichbar ist.	Unter Windows kann dies als Fehler 10054 erscheinen. Unter Linux kann es je nach System anders oder gar nicht direkt im Python-Fehler sichtbar werden.
Keine Antwort	Der Port kann offen sein und nur auf andere Daten warten, geschlossen sein oder durch eine Firewall gefiltert werden.	Das Programm läuft in einen Timeout.

Im Versuch wurde keine UDP-Antwort empfangen. Außerdem wurde auf Fedora/Linux kein Windows-Fehlercode 10054 angezeigt. Stattdessen gab das Programm für die Ports einen `TimeoutError` aus. Das bedeutet bei UDP nicht sicher, dass der jeweilige Port geschlossen ist. Bei UDP ist ein fehlendes Antwortpaket mehrdeutig. Ein offener UDP-Dienst kann ebenfalls nicht antworten, wenn die gesendete Nachricht nicht zum erwarteten Protokoll passt. Deshalb

kann aus diesem Versuch nur sicher geschlossen werden, dass im getesteten Bereich keine UDP-Antwort auf die gesendete Testnachricht empfangen wurde.

4.3 Echo-Dienst auf Port 7

Der Echo-Dienst wurde mit dem Programm `echo_client.py` getestet. Dieses Programm baut je nach Parameter entweder eine TCP-Verbindung auf oder sendet ein UDP-Datagramm und gibt die Antwort aus. Zuerst wurde TCP getestet:

```
[TCP CLIENT] before connect local=('0.0.0.0', 0)
[TCP CLIENT] after connect local=('141.37.204.2', 59364) peer=('141.37.
[TCP CLIENT] TX: b'Echo-Test TCP'
[TCP CLIENT] RX: b'Echo-Test TCP'
```

Der TCP-Echo-Test war erfolgreich. Der Client sendete die Nachricht `Echo-Test TCP` und empfing dieselbe Nachricht zurück.

4.3.1 Wireshark-Auswertung TCP-Echo

Der Wireshark-Mitschnitt zum TCP-Echo-Test zeigt denselben grundsätzlichen Ablauf wie der offene Port 7 im TCP-Scan, nur übersichtlicher, weil hier nur ein einzelner Dienst getestet wurde. Zuerst wird die TCP-Verbindung aufgebaut. Danach sendet der Client die Nutzdaten `Echo-Test TCP`. Der Server sendet genau diese Bytes zurück. Dadurch ist im Mitschnitt direkt erkennbar, dass es sich um einen Echo-Dienst handelt.

Die Paketfolge lässt sich so zuordnen:

Paket	Richtung	Erklärung
SYN	Client → Server:7	Der Client ruft <code>connect()</code> auf.
SYN, ACK	Server:7 → Client	Der Echo-Dienst nimmt die Verbindung an.
ACK	Client → Server:7	Verbindung ist aufgebaut.
PSH, ACK	Client → Server:7	<code>sendall()</code> überträgt <code>Echo-Test TCP</code> .
PSH, ACK	Server:7 → Client	Der Server sendet <code>Echo-Test TCP</code> zurück.
FIN/ACK	Client/Server	Die Verbindung wird nach dem Test geschlossen.

Der lokale Client-Port war im Log 59364. Im Wireshark-Mitschnitt kann man die Verbindung deshalb zusätzlich über das Tupel `141.37.204.2:59364` zu `141.37.122.107:7` wiederfinden. Der Zielport 7 bleibt konstant, der Quellport ist dagegen ein automatisch gewählter lokaler Port.

Der UDP-Echo-Test gegen `141.37.168.26:7` lieferte dagegen keine Antwort:

```
[UDP CLIENT] before send local=('0.0.0.0', 0)
[UDP CLIENT] TX: b'Echo-Test UDP'
[UDP CLIENT] after send local=('0.0.0.0', 45389)
TimeoutError: timed out
```

4.3.2 Wireshark-Auswertung UDP-Echo

Beim UDP-Echo-Test sieht man im Mitschnitt nur das ausgehende UDP-Datagramm vom Client zum Server. Der Client verwendete lokal den Port 45389 und sendete an den Zielport 7. Da UDP verbindungslos ist, gibt es davor keinen Handshake. Nach dem Senden wartete der Client mit `recvfrom()` auf eine Antwort. Da keine UDP-Antwort empfangen wurde, lief der Aufruf in einen Timeout. Im Mitschnitt bedeutet das: Nach dem Paket Client → Server:7 erscheint kein passendes Antwortpaket Server:7 → Client. Daraus kann man nicht sicher schließen, ob der UDP-Port geschlossen ist oder ob der Dienst nur nicht auf diese Nachricht geantwortet hat. Für diesen Versuch ist aber klar: Ein TCP-Echo-Dienst auf Port 7 war erreichbar, ein UDP-Echo-Dienst lieferte mit der getesteten Nachricht keine Antwort.

4.4 Ergebnis Port Scan

Die offenen TCP-Ports im Bereich 1 bis 50 waren im Versuch:

7, 9, 13, 19, 22

Für UDP wurde in diesem Versuch im Bereich 1 bis 50 kein Port gefunden, der eine Antwort geliefert hat. Daher konnte experimentell kein sicher offener UDP-Port bestimmt werden. In der Wireshark-Auswertung zeigt sich der Unterschied zwischen TCP und UDP deutlich: Bei TCP kann ein offener Port am vollständigen Verbindungsaufbau erkannt werden. Bei UDP ist eine fehlende Antwort dagegen mehrdeutig und muss vorsichtiger interpretiert werden.

5 Mail

5.1 SMTP über OpenSSL

Für den SMTP-Test wurde mit OpenSSL eine Verbindung zum ASmtp-Server der HTWG auf Port 587 aufgebaut:

```
openssl s_client -starttls smtp -crlf -connect asmtp.htwg-konstanz.de:587
```

Der Server wurde zu 141.37.20.66 aufgelöst und die TLS-Verbindung wurde erfolgreich aufgebaut. Im Log war unter anderem zu sehen:

```
CONNECTED(00000003)
Verification: OK
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Verify return code: 0 (ok)
```


Nach AUTH LOGIN wurden Benutzername und Passwort Base64-kodiert übertragen. Diese Werte werden hier nicht abgedruckt. Der Login war erfolgreich:

```
334 VXNlcm5hbWU6
334 UGFzc3dvcmQ6
235 2.7.0 Authentication successful
```

Danach wurde eine Mail an meinen eigenen Gmail-Account gesendet. Die SMTP-Kommandos waren im Prinzip:

```
ehlo localhost
auth login
<BASE64_USERNAME>
<BASE64_PASSWORD>
mail from:<pa871kru@htwg-konstanz.de>
rcpt to:<kruesselpaul@gmail.com>
data
from: pa871kru@htwg-konstanz.de
to: kruesselpaul@gmail.com
subject: RN Labor OpenSSL Test
```

Dies ist eine Testmail per OpenSSL.

```
.
quit
```

Der Server nahm die Mail an:

```
250 2.1.0 Ok
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as 4D81D100037
221 2.0.0 Bye
```

Damit wurde gezeigt, dass die Mail über SMTP mit STARTTLS und Authentifizierung verschickt werden konnte.

5.2 Test mit anderem Absender

Anschließend wurde noch eine Mail an meinen eigenen Account gesendet, bei der der Envelope-Absender und der sichtbare Header-Absender willkürlich gesetzt wurden. Die sensiblen Login-Daten werden auch hier nicht abgedruckt.

```
mail from:<beliebig@example.com>
rcpt to:<kruesselpaul@gmail.com>
data
from: Fantasie Name <fake.absender@example.com>
to: kruesselpaul@gmail.com
subject: RN Labor Fake From Test
```

Dies ist ein kontrollierter Test an mich selbst.

```
.
quit
```

Auch diese Mail wurde vom SMTP-Server angenommen:

```
235 2.7.0 Authentication successful
250 2.1.0 Ok
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as 2B82A100037
221 2.0.0 Bye
```

Auffällig ist dabei, dass der sichtbare Absender im Mailprogramm aus dem `from:`-Header stammt und nicht zwingend dem Benutzer entspricht, der sich am SMTP-Server authentifiziert hat. Das zeigt, warum man Absenderangaben in E-Mails nicht blind vertrauen sollte. Moderne Mailserver und Mailprogramme können solche Mails zusätzlich durch SPF, DKIM oder DMARC bewerten und gegebenenfalls als verdächtig markieren.

5.3 SMTP in Python

Für den Python-Versuch wurde das Programm `smtp_socket_client.py` geschrieben. Es verwendet keine Mail-Bibliothek wie `smtplib`. Stattdessen werden direkt ein TCP-Socket und danach ein TLS-Socket verwendet. Der genaue Ausführungsbefehl ist im Abschnitt zu den verwendeten Python-Programmen aufgeführt. Der Ablauf ist:

1. TCP-Verbindung zu `asmtplib.htwg-konstanz.de:587` öffnen
2. EHLO senden
3. STARTTLS senden
4. Socket mit `ssl.create_default_context()` in einen TLS-Socket umwandeln
5. erneut EHLO senden

6. mit AUTH LOGIN anmelden

7. mail from, rcpt to, data und quit senden

Der Verbindungsaufbau und der Wechsel auf TLS funktionierten im Versuch:

```
[INFO] connect ('asmtplib3.htwg-konstanz.de', 587)
[S] 220 mailgate3.htwg-konstanz.de ESMTP Postfix (Ubuntu)
[C] ehlo localhost
[S] 250-mailgate3.htwg-konstanz.de
250-STARTTLS
[C] starttls
[S] 220 2.0.0 Ready to start TLS
[INFO] TLS established
[C] ehlo localhost
[S] 250-mailgate3.htwg-konstanz.de
250-AUTH PLAIN LOGIN
```

Die Login-Daten wurden im Skript maskiert ausgegeben:

```
[C] auth login
[S] 334 VXNlcm5hbWU6
[C] ***
[S] 334 UGFzc3dvcmQ6
[C] ***
```

In den aufgezeichneten Python-Läufen ist die Authentifizierung allerdings fehlgeschlagen:

```
[S] 535 5.7.8 Error: authentication failed: authentication failure
```

Danach wurde der Empfänger vom Server abgelehnt, da ohne erfolgreiche Authentifizierung kein Versand an externe Empfänger erlaubt war:

```
[S] 554 5.7.1 <kruesselpaul@gmail.com>: Recipient address rejected: Acc
[S] 554 5.5.1 Error: no valid recipients
```

Der Python-Client zeigt damit trotzdem den Protokollablauf bis einschließlich STARTTLS und AUTH LOGIN. Für einen erfolgreichen Versand müsste der Python-Lauf noch einmal mit den korrekten Zugangsdaten wiederholt werden. Der OpenSSL-Versuch zeigt, dass der SMTP-Server grundsätzlich erreichbar ist und der Versand mit gültiger Authentifizierung funktioniert.